Đồ án tốt nghiệp RQ-C1

Evidence-based CVE Applicability

Tìm hiểu sơ bộ và kế hoạch

Bản cập nhật 06/10/2026

---

1. Tóm tắt

Đề tài đặt câu hỏi: *kết hợp nhiều nguồn bằng chứng có nâng cao độ chính xác khi xác định một CVE có thực sự áp dụng cho một triển khai web cụ thể hay không*. Đồ án chọn hướng **kiểm chứng khả năng áp dụng CVE mà không trực tiếp khai thác mục tiêu**, kết luận ở **mức 2** (áp dụng cho triển khai cụ thể, kèm cấu hình/tiền điều kiện). Đối tượng ưu tiên là **CMS/framework/webmail**; việc tái tạo lỗi trong môi trường sandbox được dùng làm căn cứ để xác lập nhãn chuẩn, với mục tiêu ≥ 30 **CVE khác nhau**.

2. Hiểu bài toán

Pipeline chính của đề tài:

`input (product + version)` $\rightarrow$ `thu bằng chứng (CPE/advisory/version history/patch + quan sát HTTP)` $\rightarrow$ `evidence-fusion` $\rightarrow$ `Applicable / Not Applicable / Unknown` $\rightarrow$ `đánh giá`

Ba điểm mấu chốt:

- **Ba mức kết luận khác nhau** (mục 2.1): "CVE ảnh hưởng bản phần mềm" $\neq$ "CVE áp dụng cho triển khai cụ thể" $\neq$ "đã khai thác thành công". Đồ án chọn mức 2.
- **Unknown là đầu ra hợp lệ.** Thiếu bằng chứng thì trả Unknown, không được coi "không chắc" thành "Not Applicable".
- **Vì sao cần kết hợp nhiều nguồn.** Một nguồn đơn (CPE hay chuỗi version) không đáng tin: range CPE có thể thiếu hoặc sai, tên sản phẩm mơ hồ, và distro có thể *backport* bản vá khiến version string "có vẻ affected" nhưng thực chất đã vá. Kết hợp nhiều nguồn giúp giảm false mapping.

2.1. Ba mức kết luận (không được trộn lẫn)

Mức kết luận	Câu hỏi trả lời	Bằng chứng cần
1. Bản phần mềm (version)	Bản này có chứa lỗi CVE, hay đã vá?	product, version/build, advisory, patch
2. Triển khai cụ thể (deployment)	Cấu hình/điều kiện thực tế có thỏa để lỗi gây tác động?	thêm config, preconditions
3. Đã khai thác thành công	Có thực sự khai thác được không?	runtime evidence (PoC chạy)

Ghi chú input: pipeline ghi input là (`product + version`) — tương ứng mức 1. Ở mức 2 (deployment), input mở rộng thêm bằng chứng cấu hình/tiền điều kiện *quan sát được qua HTTP* (không dùng cấu hình nội bộ làm đầu vào của phương pháp).

3. Định nghĩa và phạm vi đối tượng nghiên cứu

3.1. Thuật ngữ

- **Vulnerability** (NIST SP 800-30 Rev. 1, dẫn qua CNSSI 4009): *"Weakness in an information system, system security procedures, internal controls, or implementation that could be exploited or triggered by a threat source."* [1]
- **Vulnerability** (OWASP): *"A weakness that makes the system susceptible to attack or damage."* [2]
- **System** (OWASP): *"A generic term covering the operating systems, web server, application frameworks and related infrastructure."* [2]

3.2. Phạm vi "web interface"

Không có chuẩn duy nhất liệt kê sẵn "web interface". Đồ án dùng định nghĩa **operational** (lựa chọn của tác giả, dẫn dắt từ định nghĩa "System" của OWASP [2]): **web-facing target** = phần mềm dùng giao thức HTTP(S) và quan sát/phát hiện được từ phía mạng, gồm 4 nhóm:

Nhóm	Ví dụ
1. Web server	Apache HTTP Server, nginx, IIS, Tomcat
2. Reverse proxy / LB / WAF / gateway	HAProxy, Traefik, Envoy, Kong
3. Backend framework	Spring, Django, Rails, Laravel, Express.js
4. Web application / CMS	WordPress, Drupal, Joomla, Jenkins, Grafana

Ngoài phạm vi (không nói HTTP hoặc là hạ tầng bên dưới): SSH (OpenSSH), Docker daemon, cơ sở dữ liệu, kernel hệ điều hành, DNS, FTP.

Ghi chú: việc chia 4 nhóm là *lựa chọn operational* để giới hạn dataset; không phải chuẩn công bố.

4. Bài học từ hai bài báo

Hai bài báo bổ trợ cho cách nghiên cứu; **không bài nào** làm đúng bài toán của đồ án.

4.1. VAVI (ASE 2025) — Vulnerability-Affected Versions Identification [4]

Thực chất: nghiên cứu thực nghiệm đánh giá 12 công cụ xác định "phiên bản bị ảnh hưởng" trên 1.128 CVE C/C++ (9 dự án). Đây là bài toán *mức 1* (affected version), không phải web fingerprinting.

Kết quả cần đọc đúng:

- Công cụ đơn tốt nhất (VCCFinder): accuracy **44,9%** ở mức *đúng toàn bộ tập affected versions* (exact-set), F1 **77,8%** ở mức từng version.
- Kết hợp nhiều công cụ (ensemble voting): accuracy **55,0%**, F1 **84,8%**.

Lưu ý: **44,9% là accuracy của *một công cụ* (single tool), không phải "một nguồn bằng chứng" (single source).** Không được dùng con số này dự báo baseline CPE-only của đồ án, cũng không suy sang web black-box.

Điều dùng được: (i) chuẩn gán nhãn *độc lập* (2 người + phân xử, Cohen's Kappa 0,83); (ii) metric hai tầng (exact-set + per-unit); (iii) bài học: nhận diện affected version còn sai nhiều, nhưng đó là mức 1 — đồ án nhắm mức 2.

4.2. Vuln2Action (JISA 2026) [5]

Thực chất: framework dựa LLM biến mô tả CVE thành *chuỗi bước tái tạo* (dạng văn bản) và map sang exploit có sẵn. Không chạy PoC thật; đánh giá chủ yếu là tương đồng văn bản (BLEU 0,7589) và review chuyên gia (86,9% correctness). Sandbox execution nằm *ngoài phạm vi*.

Lưu ý diễn giải đúng: "thiếu kiểm chứng thực thi" là giới hạn có thật của Vuln2Action. Tuy nhiên, lab của đồ án kiểm chứng *nhãn applicability cho classifier*, khác với kiểm chứng *bước do Vuln2Action sinh ra*. Tính mới phải được lập luận từ bài toán, cách biểu diễn bằng chứng, cách kết hợp và cách đánh giá của riêng đồ án.

Điều dùng được: mẫu kết hợp nhiều nguồn CTI (CVE/CWE/CPE/CAPEC/Exploit-DB) và danh sách nguồn bằng chứng.

5. Hướng đi đã chọn

Đồ án chọn:

- **Kiểm chứng khả năng áp dụng mà không trực tiếp khai thác**: kết hợp nhiều nguồn bằng chứng → quyết định `Applicable / Not Applicable / Unknown`, so sánh với baseline dùng nguồn lẻ trên cùng benchmark có nhãn chuẩn độc lập.
- **Mức kết luận 2** (deployment): "khả thi" được hiểu là *applicable* (thỏa cấu hình/tiền điều kiện), *không phải* exploitable (mức 3). Lý do: mức 1 chỉ cần (product, version), gần trùng bài toán affected-version đã được nghiên cứu; mức 2 mới là nơi kết hợp nhiều nguồn (gồm cả bằng chứng cấu hình) tạo khác biệt so với nguồn đơn.
- **Đối tượng ưu tiên**: CMS/framework/webmail (nơi có nhiều CVE và có thể tái tạo được); web server như Apache/nginx giữ vai trò phụ.
- **Tái tạo trong sandbox là bắt buộc** để xác lập nhãn chuẩn — không tái tạo được thì không có ground truth. Mục tiêu ≥ 30 **CVE khác nhau**.

Câu hỏi nghiên cứu cụ thể: *"So với dùng một nguồn bằng chứng lẻ (baseline), kết hợp nhiều nguồn có làm tăng accuracy/precision/recall và giảm false mapping hay không, và tăng/giảm bao nhiêu (đo trên benchmark có nhãn chuẩn độc lập)?"*

6. Phương pháp nghiên cứu và ground truth

6.1. Đơn vị đánh giá và nhãn

Mỗi **case** = bộ tứ (`product, version/build, configuration, CVE`). Ở mức 2, *configuration* là một phần định danh case (cùng version khác config → khác case).

- **Nhãn chuẩn (ground truth)**: `Applicable / Not Applicable` + bucket `Unresolved` (chưa xác định được trạng thái thật). `Unresolved` không phải lớp mục tiêu.
- **Đầu ra phương pháp**: `Applicable / Not Applicable / Unknown`.

6.2. Ground truth hai lớp (tách khỏi đầu vào phương pháp)

1. **Lớp A — dải affected (mức 1):** vendor advisory + patch reference (NVD/CPE). Nguồn công khai.
2. **Lớp B — applicability (mức 2): tái tạo trong sandbox** — dựng đúng (product, version, config), chạy tiền điều kiện + PoC có kiểm soát, ghi kết quả.

Luật nhãn âm (hai nhánh):

- (a) danh tính bản phần mềm + dải affected *đáng tin* và version nằm ngoài dải → Not Applicable.
- (b) bản bị ảnh hưởng nhưng *mỗi đường tác động liên quan đều thiếu ít nhất một điều kiện bắt buộc* → Not Applicable (không cần mọi điều kiện đều vắng; nếu nhiều đường phải kiểm từng đường).

Chống circularity: người gán nhãn không xem dự đoán của phương pháp; PoC thất bại không tự sinh nhãn âm; mỗi nhãn kèm vết truy (advisory + artifact tái tạo + config diff).

6.3. Đo lường (Unknown vs sai)

Unknown là *từ chối kết luận trung thực* khi thiếu bằng chứng — không tự là sai, nhưng là một *bỏ lỡ* cần đo. Quy ước metric:

- `coverage` = số dự đoán Applicable/Not Applicable trên số ca có nhãn chuẩn xác định.
- Unknown `rate` (cùng mẫu số); `precision` trên các dự đoán Applicable.
- `recall(Applicable)` giữ *toàn bộ* ground-truth Applicable ở mẫu số, kể cả ca bị trả Unknown (không bỏ Unknown để báo recall cao hơn).

6.4. Đếm CVE khác case

Theo dõi riêng `n_unique_cves` (mục tiêu ≥ 30), `n_deployments`, `n_cases`, `n_observation_conditi`
Biến thể config không làm tăng số CVE; các case cùng CVE phụ thuộc nhau (không chia ngẫu nhiên vào tập chỉnh luật và tập đánh giá).

6.5. Các bước thực hiện

1. **Thí điểm** 3 ca đại diện (trước khi mở rộng) để kiểm chứng cả khả năng tái tạo lẫn khả năng quan sát tiền điều kiện qua HTTP.
2. **Xây benchmark**: chọn sản phẩm/plugin + registry ≥ 30 CVE khác nhau; tái tạo từng ca để gán nhãn chuẩn.
3. **Trích xuất bằng chứng**: CPE/NVD, vendor advisory, version history, patch + dấu hiệu HTTP (mức 2).
4. **Baseline đơn nguồn** và **fusion** (rule-based/voting) → 3 lớp.
5. **Đánh giá** trên cùng benchmark: accuracy, precision, recall, false mapping, coverage, Unknown rate.
6. **Phân tích** + threats to validity.

7. Phạm vi sơ bộ

- **Sản phẩm:** CMS (WordPress + một số plugin, Drupal), webmail (Roundcube) là chính; Apache/nginx giữ ít ca. Framework tạm gác giai đoạn này.
- **Nguồn bằng chứng:** CPE/NVD, vendor advisory, version history, patch reference + *dấu hiệu tiền điều kiện quan sát qua HTTP* ở mức 2. Source/binary là phần mở rộng nếu còn thời gian.
- **Lab tái tạo:** bắt buộc, ≥ 30 CVE; dựng affected/fixed + biến thể config để có ca dương và âm đã kiểm chứng.
- **Tool:** prototype pipeline (collector → fusion → evaluation), fusion rule-based/weighted.
- **Đánh giá:** accuracy, precision, recall, false mapping, coverage, Unknown rate.

8. Kế hoạch (tháng 10 – 12/2026)

Giai đoạn	Thời gian	Nội dung chính
1. Thí điểm	Tuần 1–2	3 ca: Really Simple Security CVE-2024-10924, Drupal CVE-2019-6340, Roundcube XSS
2. Benchmark + ground truth	Tuần 2–4	Chọn ≥ 30 CVE, tái tạo gán nhãn, ghi registry
3. Baseline + fusion	Tuần 4–6	Xây baseline đơn nguồn và fusion model
4. Đánh giá	Tuần 7–8	Đo accuracy/precision/recall/false mapping/coverage/Unknown
5. Viết luận văn	Tuần 8–12	Viết song song, hoàn thiện cuối T12

Tài liệu tham khảo

References

[1] NIST Computer Security Resource Center (CSRC) Glossary, mục “Vulnerability” (NIST SP 800-30 Rev. 1, dẫn qua CNSSI 4009). https://csrc.nist.gov/glossary/term/vulnerability

[2] OWASP Secure Coding Practices Quick Reference Guide, Appendix B: Glossary. https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/stable-en/03-appendices/05-glossary

[3] CISA, Known Exploited Vulnerabilities (KEV) Catalog. https://www.cisa.gov/known-exploited-vulnerabilities-catalog

[4] X. Chen, C. Liu, J. Cao, Y. Xiao, et al., “Vulnerability-Affected Versions Identification: How Far Are We?”, in *Proc. IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025. https://arxiv.org/abs/2509.03876

[5] S. Kumari and G. Yadav, “Vuln2Action: An LLM-based framework for generating vulnerability reproduction steps and mapping exploits”, *Journal of Information Security and Applications*, vol. 99, 2026, art. 104420. https://doi.org/10.1016/j.jisa.2026.104420